

# CloudWatch

Server Resource Anomaly Detection System

Domain & Technical Architecture Research Report

---

Final Project – Pre-Implementation Research Deliverable

Domain: IT Infrastructure Monitoring & AIOps

Stack: Flask · SQLite · HTML/CSS/JavaScript · Machine Learning

July 2026

# Introduction

---

CloudWatch is a web-based server resource monitoring and anomaly-detection system. It continuously collects resource-utilization metrics (CPU, memory, disk, and network) from one or more servers, stores the data, and applies a machine-learning model to flag unusual behaviour before it turns into downtime. This report builds the technical foundation for that project: it studies the monitoring domain, reviews existing industry solutions, examines the Flask/SQLite/JavaScript stack the project will be built on, and lays out the machine learning approach and system design that will guide implementation.

## Part 1 – Domain Research

---

### 1.1 Domain Introduction

CloudWatch sits in the **IT infrastructure monitoring / observability** domain, a branch of DevOps and Site Reliability Engineering (SRE) concerned with knowing the real-time health of servers, containers, and networks. The specific sub-field this project targets – automatically spotting abnormal resource usage with machine learning rather than fixed thresholds – is often called **AIOps (Artificial Intelligence for IT Operations)**.

**Why the domain matters:** almost every digital service – banking apps, e-commerce sites, streaming platforms – runs on servers whose CPU, memory, disk, and network usage must stay within safe limits. When a resource is exhausted or behaves erratically (a memory leak, a runaway process, a DDoS-style traffic spike), the service degrades or goes down. Detecting this early, and automatically, directly protects revenue, user trust, and compliance obligations.

**Current industry trends** in this space include:

- **Shift from static thresholds to ML-based baselining** – instead of a fixed "alert if CPU > 90%" rule, modern tools learn a server's normal behaviour and flag statistical deviations, cutting false alarms.
- **Unified observability** – merging metrics, logs, and traces into one platform so root-cause analysis does not require jumping between tools.
- **AI-assisted root-cause analysis and auto-remediation** – leading commercial platforms now pair anomaly detection with suggested or automated fixes rather than a bare alert.
- **Open-source vs. managed SaaS trade-off** – self-hosted stacks (Prometheus/Grafana, Zabbix) remain popular for cost and data-control reasons, while SaaS platforms (Datadog, Dynatrace) win on convenience and integration breadth.
- **Lightweight, resource-efficient agents** – a growing preference for low-overhead collectors, especially for small fleets and homelabs, instead of heavyweight enterprise agents.

### 1.2 Industry Problems

#### Current Challenges

- **Alert fatigue:** static threshold rules generate large volumes of false positives, causing operators to ignore or mute alerts – sometimes missing real incidents.
- **Reactive rather than predictive monitoring:** most legacy tools notify staff only after a resource has already breached a hard limit, leaving little time to act before an outage.
- **Fragmented tooling:** teams often stitch together a collector, a database, and a dashboard from separate open-source projects, increasing operational overhead.
- **High cost of enterprise observability platforms:** host-based and ingest-based pricing on commercial SaaS tools scales quickly and can become a major line item as a fleet grows.
- **Skill and setup burden of open-source stacks:** free tools are powerful but demand engineering time to deploy, tune, and maintain.

## Why Existing Systems Are Inefficient

Threshold-based systems cannot represent a server's normal *pattern* of behaviour – only fixed limits. A server that regularly peaks to 85% CPU during a nightly batch job will either trigger constant false alerts or force operators to set the threshold so high that genuine anomalies slip through. Rule-based systems also cannot adapt automatically as workloads change over time, requiring manual re-tuning.

## Technical Limitations of Current Solutions

- Rule engines evaluate one metric at a time and struggle to capture *combined* anomalies (e.g., CPU and network both drifting together).
- Many self-hosted stacks require multiple components (collector + time-series database + visualization layer + alerting layer), each with its own configuration and failure modes.
- Commercial AI-based anomaly detection is usually a black box, making it hard for smaller teams to understand, audit, or extend the detection logic.

## 1.3 Existing Solutions Analysis

Three widely used monitoring products were studied to understand real-world architecture and feature choices relevant to CloudWatch.

| Product              | Key Features                                                                       | Technologies Used                                                            | Advantages                                                                             | Limitations                                                                                        |
|----------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Zabbix               | Agent & agentless monitoring, trigger-based alerting, templates, event correlation | C backend, RDBMS (MySQL/PostgreSQL) storage, web frontend (PHP)              | Free and open source; very flexible; mature (20+ years); strong for on-prem/hybrid     | Steep configuration curve; UI feels dated; scaling requires DB tuning                              |
| Prometheus + Grafana | Pull-based metric scraping, PromQL query language, alertmanager, rich dashboards   | Go-based Prometheus server, time-series storage, Grafana visualization layer | CNCF standard for cloud-native/Kubernetes; expressive queries; huge exporter ecosystem | Not a single product (needs several components); long-term storage needs add-ons like Thanos/Mimir |

| Product | Key Features                                                                              | Technologies Used                                 | Advantages                                                                     | Limitations                                                                                    |
|---------|-------------------------------------------------------------------------------------------|---------------------------------------------------|--------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Datadog | Unified metrics, logs, traces, ML-based anomaly detection ("Watchdog"), 500+ integrations | SaaS platform, proprietary agent, managed backend | Fast time-to-value; best-in-class UI; automated anomaly & root-cause detection | Expensive at scale (per-host and per-GB pricing); vendor lock-in; data leaves the organization |

These three products illustrate the two dominant approaches CloudWatch draws on: the **self-hosted, database-backed architecture** of Zabbix/Prometheus (informing CloudWatch's Flask + SQLite design), and the **ML-driven anomaly scoring** popularised by Datadog's Watchdog and similar features (informing CloudWatch's use of an anomaly-detection model instead of static thresholds alone).

## 1.4 Market Research

### Target Users

- System administrators and DevOps engineers managing small-to-medium server fleets
- Small businesses and startups that cannot afford enterprise SaaS observability pricing
- Students, homelabs, and self-hosters who want lightweight resource monitoring

### Target Industries

- IT services and managed hosting providers
- E-commerce platforms with predictable but variable server load
- Educational institutions running internal servers and labs
- Any organisation operating on-premise or self-managed cloud infrastructure

### Future Demand and Growth Opportunities

Industry analysis consistently shows monitoring shifting toward AI-assisted, predictive detection rather than reactive alerting, and toward lower-cost self-hosted alternatives as SaaS pricing pressure grows. This creates room for a lightweight, ML-powered, self-hosted tool such as CloudWatch that offers anomaly detection without the licensing cost or operational complexity of enterprise platforms. Future extensions could include multi-server fleet views, container-level metrics, and integration with notification channels such as email or messaging apps.

## Part 2 – Technical Architecture Study

---

### 2.1 Flask Technology Study

#### Flask Architecture

Flask is a lightweight, WSGI-based Python micro-framework. Unlike full-stack frameworks, it provides only the essentials – routing, request handling, and templating – and leaves choices like database layer or authentication to the developer, which keeps a project like CloudWatch small and easy to reason about.

#### Request–Response Cycle

A client (browser or agent script) sends an HTTP request. The Werkzeug WSGI server routes it to the matching Flask view function based on URL and HTTP method. The view function executes application logic (e.g., saving a metrics payload, querying anomaly history), and returns a response – HTML rendered via Jinja2, or JSON for API calls – which Flask converts into an HTTP response sent back to the client.

#### Routing Mechanism

Routes are declared with the `@app.route()` decorator, mapping a URL pattern and HTTP method(s) to a Python function. CloudWatch uses routes such as `/api/metrics` (POST, receive metrics from agents), `/api/anomalies` (GET, fetch flagged anomalies), and `/dashboard` (GET, render the monitoring UI).

#### Blueprint Architecture

Blueprints let a Flask application be split into reusable modules – for example, an auth blueprint for login/session routes, an api blueprint for metric ingestion and anomaly endpoints, and a dashboard blueprint for page rendering. This keeps the codebase organised as the project grows beyond a single file.

#### Jinja Template Engine

Jinja2 renders HTML dynamically using template inheritance (a base layout extended by page templates), control structures (`{% for %}`, `{% if %}`), and auto-escaping of variables to prevent injection of unsafe HTML.

#### Session Management

Flask uses signed, cookie-based sessions (via `SECRET_KEY`) to keep a user logged in across requests without storing session state server-side by default, which is sufficient for a single-team internal monitoring tool.

#### Form Handling & API Integration Concepts

Flask reads form data via `request.form` and JSON payloads via `request.get_json()`, which is how agent scripts push metric readings to the API endpoints as structured JSON.

#### Database Connectivity

Flask connects to SQLite either through Python's built-in `sqlite3` module or an ORM such as SQLAlchemy, executing parameterised queries to insert incoming metrics and read them back for the dashboard and the ML pipeline.

## 2.2 Frontend Technology Study

### HTML

- **Semantic HTML:** using tags like <header>, <main>, <section>, and <table> for metric grids gives the dashboard a meaningful, accessible structure.
- **Form elements:** inputs for adding a new server to monitor, setting threshold overrides, and login credentials.
- **Accessibility:** label associations, ARIA roles for alert banners, and sufficient colour contrast for status indicators (normal/warning/critical).
- **Structure best practices:** separating layout (HTML), styling (CSS), and behaviour (JS) into distinct files loaded by a shared base template.

### CSS

- **Responsive design** using media queries so the dashboard works on both desktop monitors and tablets used by on-call staff.
- **Flexbox** for aligning navigation bars, cards, and status badges.
- **CSS Grid** for the multi-metric dashboard layout (CPU, memory, disk, network panels).
- **CSS variables** for a consistent colour palette across normal/warning/critical states.
- **Animation basics** (transitions) for smooth chart updates and alert-banner appearances.
- **UI design principles:** clear visual hierarchy, consistent spacing, and colour-coded severity so anomalies are noticed at a glance.

### JavaScript

- **DOM manipulation** to update chart values and status badges as new data arrives.
- **Event handling** for interactions like selecting a server, filtering the time range, or acknowledging an alert.
- **Form validation** before submitting new server or threshold configuration.
- **Fetch API / AJAX** to poll /api/metrics and /api/anomalies periodically without reloading the page.
- **Dynamic rendering** of charts (e.g., with a lightweight charting library) driven by the JSON returned from Flask.

## 2.3 Database Study

### SQLite Architecture

SQLite is a serverless, file-based relational database engine – the entire database lives in one file with no separate server process, which makes it ideal for a self-contained project like CloudWatch during development and small-scale deployment.

## CRUD Operations

CloudWatch performs: **Create** (insert new metric rows and anomaly records), **Read** (dashboard queries for recent metrics and alert history), **Update** (mark an alert as acknowledged/resolved), and **Delete** (purge metrics older than a retention window).

## Database Design Principles, Relationships & Normalization

Tables are normalised to third normal form: server details, metric readings, anomaly scores, and alerts are kept in separate tables linked by foreign keys, avoiding duplicated server metadata on every metric row (full schema in Part 5).

## Security Practices

- Parameterised queries / ORM usage to prevent SQL injection.
- Hashing passwords (e.g., with Werkzeug's `generate_password_hash`) rather than storing them in plain text.
- Restricting the API's write endpoints to authenticated agents/users.
- Regular backups of the SQLite file and least-privilege file permissions.

## Part 3 – Domain-Specific Technical Study

---

As an infrastructure-monitoring project, CloudWatch's domain-specific study focuses on how monitoring data flows through a real system, and how it must be secured.

### Monitoring Data Flow

A lightweight collector script (using a library such as `psutil`) runs on each monitored server, samples CPU, memory, disk, and network counters at a fixed interval, and pushes them as JSON to the Flask API. Flask persists the raw reading, then triggers preprocessing and scoring by the ML engine. The result (normal or anomalous, with a confidence score) is written back to the database and surfaced on the dashboard in near real time.

### System & Data Security

- Metric-ingestion endpoints authenticated with an API key or token per registered server.
- HTTPS/TLS in production to protect metrics and credentials in transit.
- Role-based access so only authorised users can view or modify server configuration.
- Audit logging of configuration changes (threshold edits, server additions/removals).

### Prediction / Alerting Systems

Once the ML engine scores a metric window, CloudWatch follows a two-tier response: a **warning** state for moderate anomaly scores (visual flag on the dashboard) and a **critical** state for high-confidence anomalies (persisted alert requiring acknowledgement). This mirrors severity tiers used in commercial platforms while remaining simple enough for a single-team deployment.

### Relevant Regulations & Standards

While CloudWatch itself does not process regulated personal data, if deployed for organisations subject to data-protection or industry compliance rules (e.g., logging access to production infrastructure), it should support audit trails and configurable data-retention periods consistent with general IT governance practice.



## Part 4 – Machine Learning Research

---

| Aspect                      | Details                                                                                                                                                        |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Problem Type                | Unsupervised anomaly detection over multivariate time-series (CPU, memory, disk, network)                                                                      |
| Dataset Used                | Historical metric readings collected by CloudWatch's own agents; optionally bootstrapped with a public server-metrics dataset for initial model tuning         |
| Features Used               | Rolling-window mean, standard deviation, min/max of CPU %, memory %, disk I/O, and network throughput; rate-of-change between consecutive samples              |
| Data Preprocessing          | Missing-value handling, min-max / standard scaling, resampling to fixed time intervals, rolling-window feature construction                                    |
| Algorithms Used             | Isolation Forest as the primary unsupervised detector; a statistical baseline (rolling mean $\pm$ k·std / Z-score) as a lightweight secondary check            |
| Model Evaluation Metrics    | Precision, recall, F1-score against manually labelled anomaly windows; false-positive rate (critical for avoiding alert fatigue); ROC-AUC for threshold tuning |
| Prediction Workflow         | New metric window → feature extraction → scaling with saved scaler → model.predict/score → anomaly flag + score returned to Flask                              |
| Confidence Score Generation | Isolation Forest's anomaly score (path-length based) is normalised to a 0–1 confidence value; scores above a tuned cutoff trigger an alert                     |

### Why Isolation Forest

Isolation Forest is well suited to this project because it does not require labelled anomaly examples (rare in practice), scales well to the modest data volumes a small server fleet produces, and naturally handles multivariate input (CPU, memory, disk, network together) rather than evaluating each metric in isolation – directly addressing the "combined anomaly" limitation identified in Part 1.

## Part 5 – System Design

---

### 5.1 Architecture Diagram

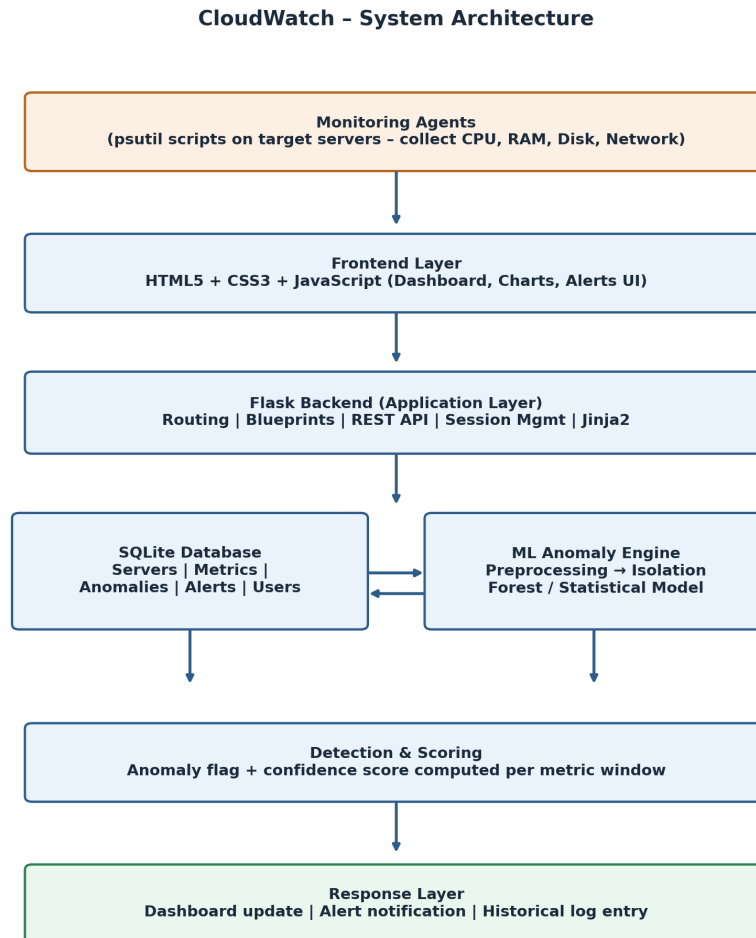


Figure 1. CloudWatch layered system architecture.

### 5.2 Workflow Diagram

## CloudWatch - Operational Workflow

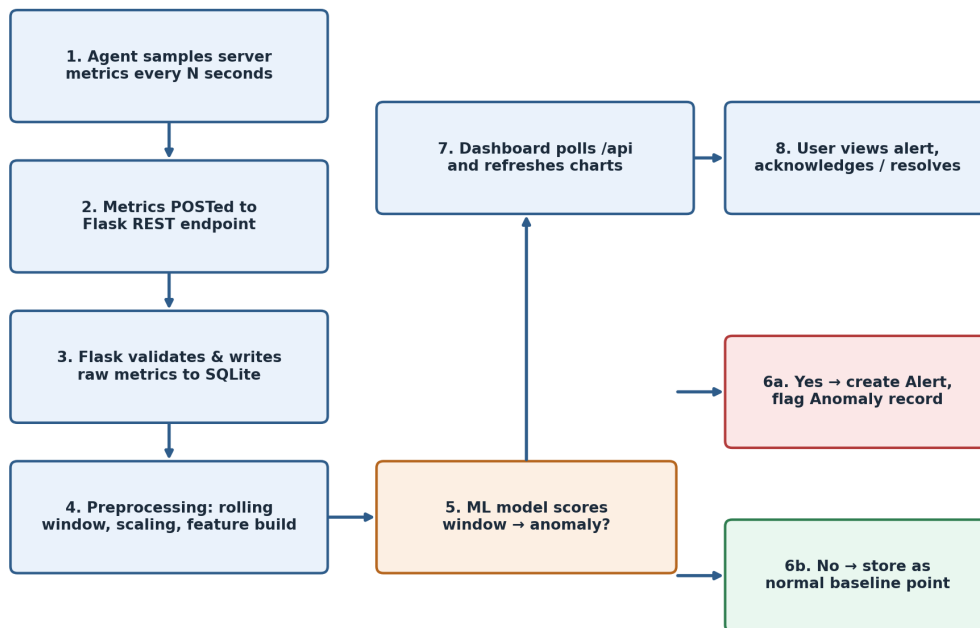


Figure 2. End-to-end operational workflow, from metric collection to alert resolution.

## 5.3 Database Design

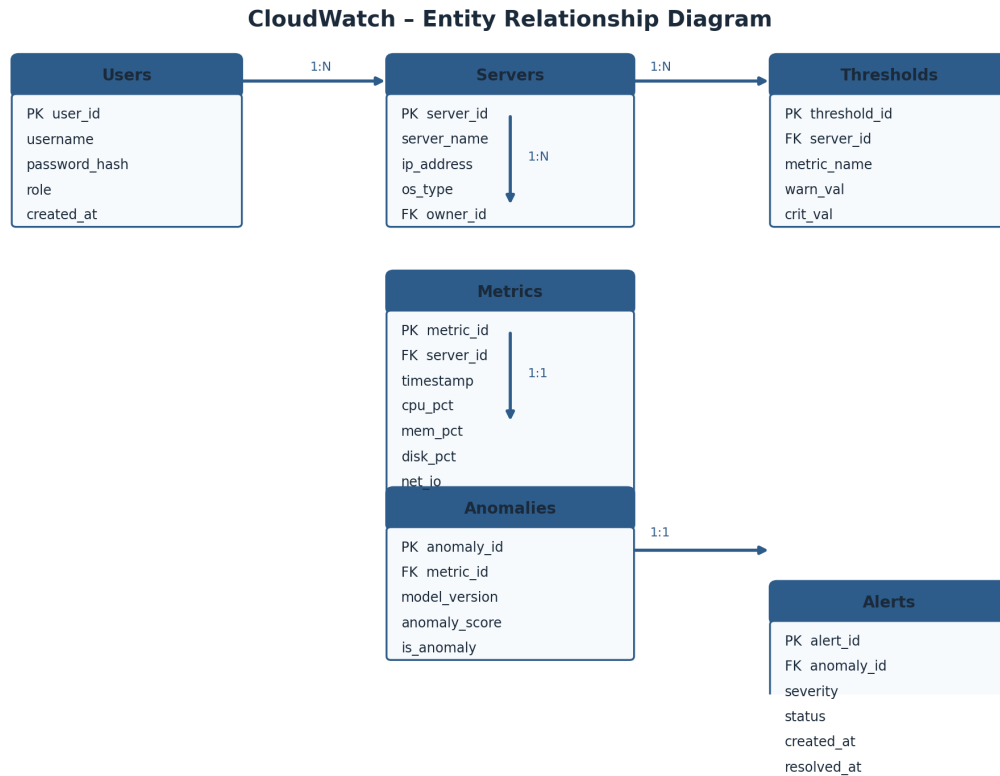


Figure 3. Entity Relationship Diagram for the CloudWatch database.

### Tables Required

| Table      | Purpose                                           | Key Columns                                                                   |
|------------|---------------------------------------------------|-------------------------------------------------------------------------------|
| Users      | Application users/administrators                  | user_id (PK), username, password_hash, role                                   |
| Servers    | Registered monitored servers                      | server_id (PK), server_name, ip_address, os_type, owner_id (FK)               |
| Metrics    | Raw resource readings                             | metric_id (PK), server_id (FK), timestamp, cpu_pct, mem_pct, disk_pct, net_io |
| Anomalies  | ML scoring results per metric window              | anomaly_id (PK), metric_id (FK), model_version, anomaly_score, is_anomaly     |
| Alerts     | Actionable notifications from confirmed anomalies | alert_id (PK), anomaly_id (FK), severity, status, created_at, resolved_at     |
| Thresholds | Optional manual override limits per server/metric | threshold_id (PK), server_id (FK), metric_name, warn_val, crit_val            |

### Relationships

- One **User** can own many **Servers** (1:N).
- One **Server** produces many **Metrics** readings over time (1:N).

- Each **Metric** reading is scored into at most one **Anomaly** record (1:1).
- A confirmed **Anomaly** may generate one **Alert** (1:1).
- One **Server** can have several custom **Thresholds**, one per metric type (1:N).

## References

---

- Zabbix, Prometheus, Grafana, and Datadog product documentation and comparison research (2026 industry surveys of server monitoring and observability tools).
- Flask Documentation – [flask.palletsprojects.com](https://flask.palletsprojects.com)
- Jinja2 Documentation – [jinja.palletsprojects.com](https://jinja.palletsprojects.com)
- SQLite Documentation – [sqlite.org/docs.html](https://sqlite.org/docs.html)
- scikit-learn Isolation Forest documentation – [scikit-learn.org](https://scikit-learn.org)
- MDN Web Docs – [developer.mozilla.org](https://developer.mozilla.org) (HTML, CSS, JavaScript references)